

Analysis: Draupnir

In this problem, we need to determine the number of each type of X-day ring by querying the total number of rings in existence at the end of a certain day.

Let R_1 be the number of 1-day rings, R_2 be the number of 2-day rings, ... etc. at the end of day 0. The number of X-day rings doubles at the end of every day that is a multiple of X. Thus, the total number of rings on day i is

$$R_1 * 2^i + R_2 * 2^{\lfloor i/2 \rfloor} + R_3 * 2^{\lfloor i/3 \rfloor} + R_4 * 2^{\lfloor i/4 \rfloor} + R_5 * 2^{\lfloor i/5 \rfloor} + R_6 * 2^{\lfloor i/6 \rfloor}.$$

Test set 1

In the first test set, we are allowed six queries to determine the six unknown variables. Note that if we query any day number larger than 63, then the number of 1-day rings on that day will be a multiple of 2^{63} (equivalently, it is 0 modulo 2^{63}). Similarly, if we query any day number larger than $X \cdot 63$, then the number of X-day rings will be 0 modulo 2^{63} on that day.

Thus, on day 315 ($=5 \cdot 63$), the total number of rings, modulo 2^{63} , is $R_6 * 2^{52}$ (since the number of 1-day, 2-day, ..., 5-day rings are all 0 modulo 2^{63}). Since $R_6 \leq 100$, we know that $R_6 * 2^{52}$ does not exceed 2^{63} , so we can directly determine R_6 . Then, on day 252 ($=4 \cdot 63$), the total number of rings, modulo 2^{63} , is $R_6 * 2^{42} + R_5 * 2^{50}$. Since we already know R_6 and we know this sum cannot be more than 2^{63} , we can solve for R_5 . We may continue this process by querying days 189 ($=3 \cdot 63$), 126 ($=2 \cdot 63$), 63 ($=1 \cdot 63$) and 1 to determine R_4 , R_3 , R_2 and R_1 , respectively.

Alternative

Note that since we are querying six days and attempting to solve for six variables, we may choose to query (for example) days 1, 2, ..., 6. This gives us a system of equations with six equations and six variables. Since these equations are linearly independent, we can solve this system, via [Gaussian elimination](#) for example, to get the solution.

Test set 2

The second test set requires another insight. We are only given two queries, so we must be able to solve for multiple R_i at once.

Let's think about what information we get by querying day 189 ($=3 \cdot 63$). This gives $R_6 * 2^{31} + R_5 * 2^{37} + R_4 * 2^{47}$, modulo 2^{63} . However, it is possible that there is some overlap between $R_6 * 2^{31}$ and $R_5 * 2^{37}$. For example, all other things being equal, we would not be able to differentiate between the case where $R_6=64$ and $R_5=0$ and the case where $R_6=0$ and $R_5=1$. Both of these would have 2^{37} total rings.

We must use the fact that $R_i \leq 100$. In order for the number of i -day rings to not interfere with the number of $(i-1)$ -day rings on day d , we need $2^{\lfloor d/(i-1) \rfloor} > 100 * 2^{\lfloor d/i \rfloor}$ (note that this is equivalent to $\lfloor d/(i-1) \rfloor \geq \lfloor d/i \rfloor + 7$ since $2^7 > 100$). If we ignore the modulo 2^{63} restriction, then we could solve the question in a single query of (for example) 1000. This would give us

$$R_1 * 2^{1000} + R_2 * 2^{500} + R_3 * 2^{333} + R_4 * 2^{250} + R_5 * 2^{200} + R_6 * 2^{166}.$$

Since $R_i \leq 100$, we could determine R_6 by taking this value modulo 2^{200} to get $R_6 * 2^{166}$, then by dividing by 2^{166} . We then could iteratively determine R_5 , R_4 , ..., R_1 . However, this idea does not work since the problem is modulo 2^{63} . There is no single query that can give us all of the information we need, modulo 2^{63} .

We will use one query to determine the values of R_4 , R_5 , and R_6 , followed by a second query to determine the values of R_1 , R_2 , and R_3 . We saw above that a query of 189 ($=3*63$) will not work. However, a query of (for example) 200 will work:

$$R_4 * 2^{50} + R_5 * 2^{40} + R_6 * 2^{33},$$

and then solve for each in the same way as for the 1000 case. We can then make a second query of (for example) 56:

$$R_1 * 2^{56} + R_2 * 2^{28} + R_3 * 2^{18} + R_4 * 2^{14} + R_5 * 2^{11} + R_6 * 2^9.$$

We know the value of R_4 , R_5 , and R_6 from the first step, so we may substitute those in and then solve for R_1 , R_2 , and R_3 one-by-one.

Note that the choice of 200 and 56 above were not the only possible options. In this problem, we could either guess-and-check for these values offline or (preferably) write a loop in the program to find two values that satisfy the criteria. We also need to ensure that the term with the largest exponent is not too large. For example, we cannot use a query of 250 in the first step since this gives $R_4 * 2^{62} + R_5 * 2^{50} + R_6 * 2^{41}$, because the first term ($R_4 * 2^{62}$) may be at least 2^{63} .

Analysis: Fair Fight

Test set 1

Let us call a pair (L, R) *fair* if choosing it produces a fair fight. Notice that since there are only 100 entries, there are at most $100 \cdot 101 / 2 = 5050$ possible intervals. For each interval, we may simply search for the maximum value in each array via a linear search and check if those maximum values are close enough.

Test set 2

Notice the random choice of sword in case of ties does not change whether (L, R) is fair or not, so we can further assume that, if there are ties, they are broken by choosing the sword with smallest index. To simplify the write-up below, we assume that each C_i is distinct.

Let us consider a sub-problem: for each sword i that Charles can choose, how many fair intervals (L, R) are there which Charles chooses sword i ? Let us call this value F_i . The answer to the original problem is simply the sum of the F_i s. For each interval (L, R) , there are three properties we are concerned with:

- (P1) Charles chooses sword i . That is, $L \leq i \leq R$ and $\max(C_L, C_{L+1}, \dots, C_R) = C_i$.
- (P2) Charles' sword is *good enough*. That is, $\max(D_L, D_{L+1}, \dots, D_R) \leq C_i + K$.
- (P3) Charles' sword is *too good*. That is, $\max(D_L, D_{L+1}, \dots, D_R) < C_i - K$.

So we have $F_i = (\text{\# of intervals that satisfy P1 and P2}) - (\text{\# of intervals that satisfy P1 and P3})$. These two quantities are computed very similarly since they just have a different bound on the right-hand side of the inequality in P2 and P3. We explain below just how to compute (# of intervals that satisfy P1 and P2) and leave computing (# of intervals that satisfy P1 and P3) to the reader.

Note that if an interval (L, R) satisfies P2, then any subinterval of (L, R) also satisfies P2. Similarly, if (L, R) satisfies P1, then any subinterval of (L, R) that still contains i also satisfies P1. Thus, we are really only interested in how far left L can go with $R=i$ (and how far right R can go with $L=i$). One option is to do a linear search for how far left L can go, but this is too slow. Instead, we [binary search](#) for how far away the left-endpoint is. A left-endpoint is too far left if P1 or P2 are no longer true. Otherwise, the left endpoint can be pushed further left. Once we know the furthest left L can go (let this index be L_i) and the furthest right R can go (let this index be R_i), then $(\text{\# of intervals that satisfy P1 and P2}) = (i - L_i + 1) \times (R_i - i + 1)$. Calculating the maximum element in a range can be computed efficiently using a [range minimum \(maximum\) query](#)-type data structure in $O(1)$ time per query, meaning $O(\log N)$ total time for the binary search.

In terms of time complexity, for each i , we need to perform 4 binary searches, which take $O(\log N)$ time each, for an overall $O(N \log N)$ total time. Setting up the range maximum query data structure takes an additional $O(N \log N)$ time, yielding an overall $O(N \log N)$ time for the full algorithm. There are solutions that can compute all of the required L_i and R_i values (defined above) in $O(N)$ total time by doing a couple of clever sweeps of the data to count the number of unfair intervals, but this was not needed for this problem.

Analysis: Manhattan Crepe Cart

Test set 1

Test sets 1 and 2 differ only in how large the available grid area is. In the first test set, people can only be standing in cells with coordinates between 0 and 10, inclusive, in either dimension. Moreover, we can notice that the cart can only be in a cell within this area. The cart's horizontal and vertical coordinates must both be nonnegative per the rules of a problem. Moreover, it cannot have a coordinate larger than 10 (in either dimension). Suppose, for example, that the cart had a horizontal coordinate greater than 10; then that would imply that there must be at least one person standing at horizontal coordinate 10 and facing east. Otherwise, placing the cart at horizontal coordinate 10 would be even better, per the tiebreaker rules. But the rules of the problem do not allow people at (horizontal/vertical) coordinate 10 to face (east/north).

Therefore, to solve test set 1, we can create an array to represent all of the blocks in the allowed area, and initialize each cell's value to 0. Then, for each person, we increment all of the cells of the array that they are walking toward. Finally, we find the cell of the array with the maximum value, using the tiebreaker rules as needed. This solution takes $O(P \times Q^2)$ time.

Test set 2: a quadratic solution

In problems that involve multiple dimensions, it is often worth checking whether those dimensions are independent. In this case, they are! A person heading west, for example, gives us a "vote" for the crepe cart being to the west of them, but tells us nothing at all about the north-south location of the cart. So we can solve the two dimensions as separate one-dimensional problems; the horizontal problem includes only the people heading west or east (and their horizontal positions), and the vertical problem includes only the people heading south or north (and their vertical positions). Let us consider the horizontal dimension for now; our arguments also apply to the vertical dimension.

Even if our people are widely spread out along the horizontal axis, the crepe cart can only possibly be in a limited number of horizontal positions. In fact, it must be either at position 0, or at a cell that is one cell to the east of some person. To see why, suppose that the cart is in some other cell $1 \leq C \leq Q$. Let W denote the cell one unit to the west of C . We know by assumption that W does not contain a person. But then if we move the cart from C to W , we will not be losing any votes (i.e. any person voting for C is also voting for W), and we may even gain votes (if there were people in C who were heading west). (Notice that a person does not vote for the cell they are in.) Even if we do not gain votes, our tiebreaker gets better. So we should always make this move, and, therefore, we should always move the cart west until it is at 0 or immediately to the east of someone. Observe that this might find only a locally optimal solution, but if we check all such cells, we will surely find the globally optimal solution.

These observations reduce the number of cells we need to check to $O(P)$ rather than $O(Q)$. To check a cell, we can make a linear pass over all of the people, and count whether each one is voting for that cell. One such check takes $O(P)$ time. Then we choose the cell that got the most votes, breaking a tie if needed by choosing the westernmost of the tied cells. The overall time complexity is $O(P^2)$ for the one-dimensional problem, and solving it twice (for two dimensions) is still $O(P^2)$.

Test set 2: a (nearly) linear solution

Although the above solution is fast enough to solve test set 2, we can do even better by avoiding making a linear pass over the data for each person.

We can first process our data into a set of tuples like the following: (coordinate, number of people at that coordinate facing west, number of people at that coordinate facing east). Let us denote these as (C_i, W_i, E_i) . (Remember that for the purposes of the horizontal subproblem, we are ignoring people facing north or south.) Using a hash-based dictionary, this processing takes time linear in P . As we do this, we should also determine the total numbers W and E of people facing west and east.

Then, we sort these tuples in ascending order of their first values. It is probably most convenient to use a common sorting algorithm (or one built into your language), making this step nonlinear. (We will leave a spirited discussion of which sorting algorithms are truly linear for another day.)

Once we have sorted the tuples, we start by considering cell 0 as a candidate location, and we determine the votes for that cell. We know that quantity is equal to W minus the number of people in cell 0, if any. We make a note of this number of votes and set 0 as our best candidate so far. Then, we look at the first tuple in our sorted list, which represents the people at some cell C (which might be cell 0). The number of votes for cell $C + 1$ (the cell one unit to the east of C) should be the same as the number of votes we found for cell 0, plus the number of people in cell C facing east (which is all of them in this case), minus the number of people in cell C facing west. If cell C is a better candidate, we store it and its number of votes.

We can do this for every tuple in our list to get our final answer. If we have one or more people on the eastern border, we do not need to check the cell one unit to the east of them, since (as explained at the start of the analysis) the cart can never have a coordinate larger than Q . Since the check for each entry of the tuple takes constant time, the full pass takes $O(P)$ time.